

Reporte Técnico de Actividades Práctico- Experimentales Nro. 003

1. Datos de Identificación del Estudiante y la Práctica

Nombre del estudiante(s)	- Ashley Patricia Mocha Valarezo - Abraham Daniel Maza Lapo - José Camilo Merino Morocho - Edgar Fabricio Rios Cuenca
Asignatura	Matemáticas Discretas
Ciclo	1er Ciclo
Unidad	Unidad 2: Álgebra de Boole
Resultado de aprendizaje de la unidad	Utiliza eficientemente los principios de la lógica matemática, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad.
Práctica Nro.	003 - Fase 5 y 6
Tipo	Grupal
Título de la Práctica	Operaciones Booleanas, Simplificación de Expresiones, Mapas de Karnaugh y Propiedades del Álgebra Booleana
Nombre del Docente	Mario Enrique Cueva Hurtado
Fecha	Semana 9 y 10: del 1 al 12 de junio
Horario	Lunes, Martes, jueves 07:30 – 09:30
Lugar	Aula de la Carrera de Computación / Laboratorio de Computación / Aula Virtual (Zoom)
Tiempo planificado en el Sílabo	18 horas (3 horas por sesión APE)

2. Objetivo(s) de la Práctica

- Comprender los principios fundamentales del álgebra de Boole y su relación con la lógica proposicional estudiada en la Unidad 1.
- Aplicar las operaciones booleanas básicas (AND, OR, NOT, XOR, XNOR, NAND, NOR) en la construcción y simplificación de expresiones booleanas.
- Dominar las técnicas de simplificación de expresiones booleanas: algebraica, por mapas de Karnaugh (2, 3 y 4 variables), y propiedades del álgebra booleana.



- Diseñar funciones lógicas a partir de especificaciones dadas, expresándolas en diferentes formas (suma de productos, producto de sumas) y minimizándolas.
- Relacionar el álgebra de Boole con aplicaciones prácticas en computación: circuitos lógicos, diseño digital, consultas SQL y programación.

3. Materiales, Reactivos, Equipos y Herramientas

- Pizarra y marcadores de colores
- Proyector multimedia
- Hojas de trabajo con ejercicios de operaciones booleanas y mapas de Karnaugh
- Plantillas impresas de mapas de Karnaugh (2, 3 y 4 variables)
- Guía de simplificación algebraica de expresiones booleanas
- Calculadora científica
- PC o laptop (mínimo 1 por estudiante) · Plataforma EVA de la UNL para entregas y foros
- Laboratorio de computación para prácticas · Herramientas colaborativas: Google Drive, Wiki
- Simuladores de circuitos lógicos online (Logisim, CircuitVerse) · Software de presentación (PowerPoint, Canva)
- Máximo de personas por grupo: 4-5 estudiantes

4. Procedimiento / Metodología Ejecutada

FASE 5 : Propiedades Adicionales del Álgebra de Boole (Semana 11: 15-19 junio 2026)

- **Paso 1:** El docente presenta propiedades avanzadas del álgebra de Boole: teorema del consenso dual, funciones booleanas completas (NAND/NOR como puertas universales), expansión de Shannon, y resolución booleana.
- **Paso 2:** Los estudiantes investigan y presentan de forma participativa las leyes y propiedades adicionales mediante ejemplos prácticos, conectando cada propiedad con aplicaciones reales en diseño digital y programación.
- **Paso 3:** Práctica de implementación de funciones booleanas usando únicamente compuertas NAND o NOR (puertas universales): los estudiantes diseñan circuitos equivalentes usando solo un tipo de compuerta.
- **Paso 4:** Los equipos elaboran un informe reflexivo sobre la optimización lógica en sistemas computacionales, analizando cómo la simplificación booleana reduce costos, consumo energético y huella ambiental en la fabricación de circuitos.
- **Paso 5:** Práctica integradora: cada equipo resuelve un caso aplicado a la computación (diseño de un sumador binario de 1 bit, un comparador digital, o un codificador/decodificador) aplicando todas las técnicas de simplificación aprendidas.

FASE 6: Evaluación Sumativa y Portafolio Integrador (Semana 12: 22-26 junio 2026)

- Paso 1: Los equipos entregan el portafolio integrador de la unidad que incluya: todos los talleres de simplificación booleana, mapas de Karnaugh resueltos, diseño de funciones lógicas, y el informe reflexivo sobre optimización.
- Paso 3: Sesión de socialización: los equipos presentan el caso aplicado que diseñaron (sumador, comparador, etc.), explicando el proceso de diseño, las simplificaciones realizadas y las ventajas obtenidas.
- Paso 4: Coevaluación entre equipos mediante rúbricas. Retroalimentación del docente sobre el portafolio y la presentación. Análisis de casos aplicados a la computación.

5. Resultados

Paso 2:

Ejercicio 1:

Universidad Nacional de Loja

Nombre: Ashley Patricia Macha Valverde (continúa en anexo 13)

Unidad: 8

Título: APE 3

Paso 2:

Los estudiantes investigan y presentan de forma participativa las leyes y propiedades adicionales mediante ejemplos prácticos, conectando cada propiedad con aplicaciones reales en diseño digital.

① Consenso Dual

$$F = (A+B)(\bar{A}+C)(B+C)(D+C)$$

Consenso = $(B+C)$

Eliminando = $(B+C)$

$$F = (A+B)(\bar{A}+C)(D+C)$$

Ejercicio 2:

② Expansión de Shannon

$$F(A, B, C) = AB + \bar{A}\bar{C} + \bar{B}C$$

$$\text{Evaluar } A=1 \rightarrow F(1, B, C) = (1)B + (1)\bar{C} + \bar{B}C$$

$$F(1, B, C) = B + \bar{C} + \bar{B}C$$

$$F(1, B, C) = B + \bar{B}C + \bar{C} \rightarrow \text{Ley conmutativa}$$

$$F(1, B, C) = (B + \bar{B})(C + \bar{C}) + \bar{C} \rightarrow \text{Ley distributiva, y ley de complemento}$$

$$F(1, B, C) = (1)(B + C) + \bar{C} \rightarrow \text{Ley de identidad}$$

$$F(1, B, C) = B + C + \bar{C} \rightarrow \text{Ley de complemento}$$

$$F(1, B, C) = B + 1 \rightarrow \text{Ley de identidad}$$

$$F(1, B, C) = 1$$

$$\text{Evaluar } A=0 \rightarrow F(0, B, C) = (0)B + (0)\bar{C} + \bar{B}C$$

$$F(0, B, C) = \bar{B}C$$

$$\text{Resultado final} \rightarrow F = A(1) + \bar{A}(\bar{B}C)$$

$$F = A + \bar{A}\bar{B}C$$

$$\text{Resultado final extendido} \rightarrow F = A(B + \bar{C} + \bar{B}C) + \bar{A}\bar{B}C$$

Ejercicio 3:

③ Resolución booleana

► Clausula 1: $x + y$

(O es un problema de software o de hardware)

► Clausula 2: $\bar{x} + y$

(Si no es un problema de software, entonces se reinicia).

► Clausula 3: $\bar{y} + w$

(Si es un problema de hardware, entonces se activa la alerta)

► Clausula 4: $\bar{z}$

(El sistema no se reinicia)

► Resolvente 1: Entre 2 y 4.

$$(\bar{x} + y)(\bar{z})$$

$$\bar{x}$$

► Resolvente 2: Entre resolvente 1 y 1

$$(x + y)(\bar{x})$$

$$y$$

► Resolvente 3: Entre resolvente 2 y 3

$$(\bar{y} + w)(y)$$

$$w \rightarrow \text{Se activa la alerta. } (1+0)(0+0)(0+\bar{1})(0+1) = 1$$

Paso 3:

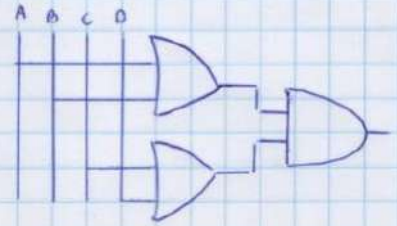
Ejercicio 1:

Paso 3: Práctica de implementación de funciones booleanas usando únicamente compuertas NAND o NOR (puertas universales): los estudiantes diseñan circuitos equivalentes usando un tipo de compuertas.

- Ejercicio 1: $F = (A+B)(C+D)$

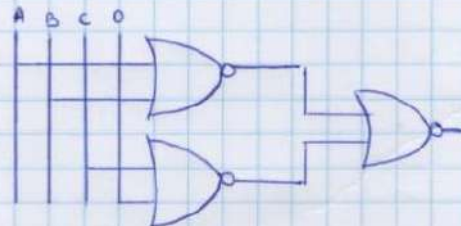
Función original

$F = (A+B)(C+D)$



Función final NOR

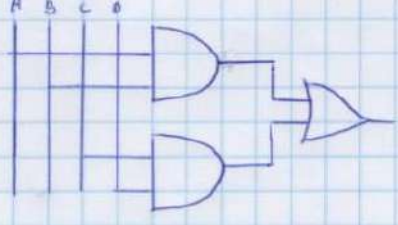
$F = (A+B)(C+D) \rightarrow$ Ley de doble negación
 $\overline{\overline{(A+B)(C+D)}} \rightarrow$ Ley de Morgan
 $\overline{(\overline{A+B}) + (\overline{C+D})}$



Ejercicio 2:

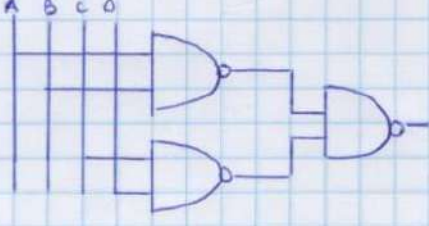
Función original

$F = AB + CD$

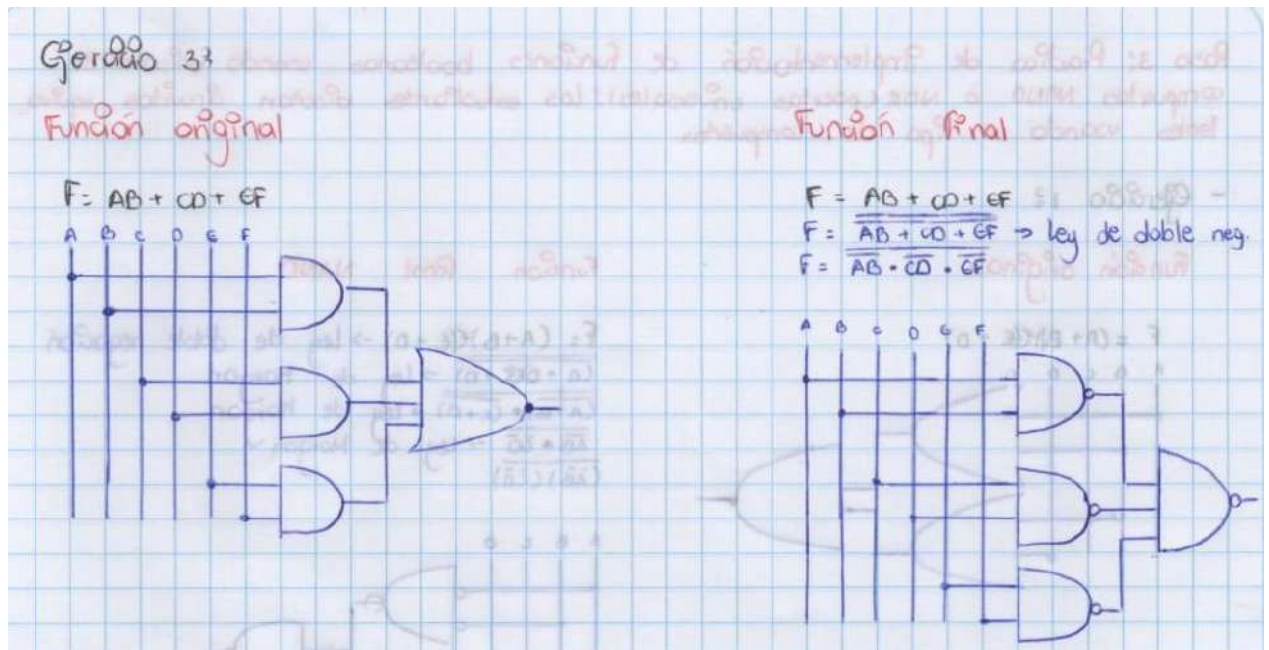


Función final NAND

$F = AB + CD \rightarrow$ Ley de doble negación
 $\overline{\overline{AB + CD}} \rightarrow$ Ley de Morgan
 $\overline{\overline{AB} \cdot \overline{CD}}$



Ejercicio 3:



Paso 4:

Paso 4:

Los equipos elaboran un informe reflexivo sobre la optimización lógica en sistemas computacionales, analizando cómo la simplificación booleana reduce costos, consumo energético y huella ambiental en la fabricación de circuitos.

Informe reflexivo sobre la optimización lógica en sistemas computacionales.

En el diseño de sistemas computacionales modernos, la eficiencia ya no se mide únicamente en términos de velocidad de procesamiento, sino también en función de la optimización de los recursos materiales y energéticos.

El presente informe expone nuestra reflexión colectiva sobre el impacto de la optimización lógica y la simplificación booleana en la ingeniería de circuitos. A lo largo de este análisis, demostramos cómo las decisiones de diseño matemático en el software abstracto se traducen directamente en beneficios económicos y ecológicos tangibles en el mundo físico.

Análisis de simplificación booleana

La simplificación booleana (mediante métodos como los Mapas de Karnaugh) permite reducir al mínimo el número de compuertas lógicas e interconexiones necesarias para ejecutar la función. Este impacto positivamente en tres críticas:

1. Reducción de Costos de Fabricación:

- Al reducir las compuertas necesarias, disminuye el área de silicio requerida para cada circuito integrado. Esto permite crear más microchips por cada oblea de producción.
- Los circuitos más pequeños y menos complejos tienen una menor tasa de defectos físicos durante el proceso de fotografía, lo que reduce el desperdicio de materiales costosos y maximiza el rendimiento (yield) de la fábrica.

2. Consumo Energético:

- Cada compuerta lógica consume energía eléctrica y disipa calor cada vez que cambia de estado. Eliminar componentes redundantes reduce la carga global y las corrientes de fuga del circuito.
- Un menor consumo de energía disminuye la temperatura operativa del dispositivo, reduciendo o eliminando la necesidad de sistemas de enfriamiento activo (como ventiladores), lo que genera un ahorro de energía secundaria.

3. Huella Ambiental:

- Circuitos más pequeños significaban una menor demanda de materias primas críticas, como silicio de alta pureza, cobre y tierras raras, mitigando el impacto de la minería intensiva.
- La reducción del tamaño de los componentes disminuye el uso de químicos peligrosos en las etapas de grabado (etching) y lavado en las plantas.

de semiconductores

- A largo plazo, millones de dispositivos optimizados operando globalmente reducen de manera significativa la demanda eléctrica y las emisiones indirectas de CO₂.

Conclusion Reflexiva

Como equipo, concluimos que la optimización de sistemas lógicos no es solo un ejercicio de eficiencia técnica, sino una práctica de ingeniería responsable. La simplificación de una educación booleana en la fase de diseño de software tiene un impacto en el ecosistema, demostrando que la optimización matemática es una de las herramientas más limpias para reducir costos industriales y mitigar la huella ecológica de la era digital.

Paso 5:

Paso 5: Caso aplicado a la computación: sumador binario de 1 bit

1. Tabla de verdad

Entradas: A, B y acarreo de la entrada (C_{in})

Salidas: S (suma) y acarreo de la salida (C_{out})

A	B	C _{in}	S	C _{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

2. Simplificación de la Suma

2.1. SOP:

$$S = \bar{A}\bar{B}C_{in} + \bar{A}B\bar{C}_{in} + A\bar{B}\bar{C}_{in} + ABC_{in}$$

$$\bar{A}\bar{B}C_{in} + \bar{A}B\bar{C}_{in} + A\bar{B}\bar{C}_{in} + ABC_{in} \rightarrow \text{Ley Distributiva}$$

$$\bar{A}(\bar{B}C_{in} + B\bar{C}_{in}) + A(\bar{B}\bar{C}_{in} + BC_{in}) \rightarrow \text{XOR definición}$$

$$\bar{A}(B \oplus C_{in}) + A(\bar{B} \oplus C_{in}) \rightarrow \text{XNOR definición}$$

$$\bar{A}(B \oplus C_{in}) + A(\overline{B \oplus C_{in}}) \rightarrow \text{XNOR = complemento de XOR}$$

$$\bar{A}(B \oplus C_{in}) + A(\overline{B \oplus C_{in}}) \rightarrow \text{definición XOR}$$

$$A \oplus (B \oplus C_{in})$$

$$A \oplus B \oplus C_{in}$$

Definiciones:

$$\text{XOR} = X \oplus Y = \bar{X}Y + X\bar{Y}$$

$$\text{XNOR} = X \odot Y = \bar{X}\bar{Y} + XY$$

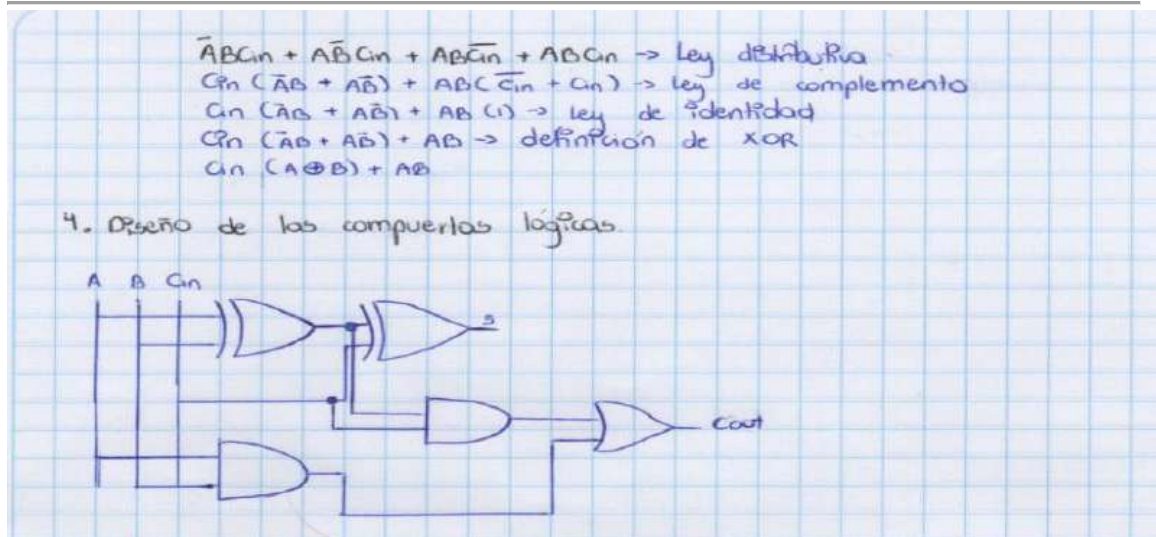
XNOR es complemento de XOR
por lo tanto se puede escribir
la función como:

$$\overline{(X \oplus Y)} = X \odot Y$$

3. Simplificación de la salida de acarreo (C_{out})

3.1. SOP:

$$C_{out} = \bar{A}BC_{in} + \bar{A}B\bar{C}_{in} + A\bar{B}\bar{C}_{in} + ABC_{in}$$



PORTAFOLIOS:

a) Edgar Fabricio Rios Cuenca

https://github.com/riosfabricio444-ai/Portaflio_Matematicas_Discretas/blob/main/UNIDAD2.md

b) Jose Camilo Merino Morocho

<https://github.com/JoseCamilo667/portafolioDigitalMD/blob/main/Unidad2.md>

c) Ashley Patricia Mocha Valarezo

<https://github.com/ashm-crypto/PortafolioMD/blob/main/Unidad2.md>

d) Abraham Daniel Maza Lapo

https://drive.google.com/drive/folders/1BR4TiEgb8dygx9DMGZ2xHWFBEKhz8-M8?usp=drive_link

6. Preguntas de Control

1. Simplifique la siguiente expresión booleana utilizando el método de Karnaugh (4 variables): $F(A,B,C,D) = \sum m(0,1,2,5,6,7,8,9,10,14)$. Muestre el mapa de Karnaugh, los grupos formados y la expresión simplificada resultante. Compare el resultado con la simplificación algebraica.

- Mapa de Karnaugh

CDAB	00	01	11	10
00	1	0	0	1
01	1	1	0	1
11	0	1	0	0
10	1	1	1	1

$$F = B'D' + B'C' + CD' + A'BD$$

- Simplificación algebraica

$$F = (ABCD)' + (ABC)'D + (AB)'CD' + A'BC'D + A'BCD' + A'BCD + A(BCD)' + A(BC)'D + AB'CD' + ABCD' - \text{Ley de complemento y distributiva}$$

$$F = (BCD)' + (BC)'D + B'CD' + A'BC'D + A'BCD' + A'BCD + ABCD' - \text{Ley de Morgan}$$

$$F = (BD)' + (BC)' + B'CD' + BCD' + A'BD - \text{Ley distributiva}$$

$$F = B'D' + B'C' + CD' + A'BD$$

- 2. Explique por qué las compuertas NAND y NOR se consideran "universales". Demuestre cómo implementar las operaciones AND, OR y NOT utilizando únicamente compuertas NAND. ¿Qué implicaciones tiene esto en el diseño de circuitos integrados?**

Las compuertas NAND y NOR se consideran compuertas universales porque, por sí solas, son capaces de implementar cualquier función lógica que pueda expresarse mediante el álgebra de Boole. Esto significa que, a partir de combinaciones adecuadas de compuertas NAND o únicamente de compuertas NOR, es posible construir las operaciones básicas AND, OR y NOT, así como circuitos digitales más complejos.

Esta característica tiene una gran importancia en el diseño de circuitos integrados, ya que permite estandarizar los procesos de fabricación utilizando un solo tipo de compuerta lógica. Como resultado, se simplifica el diseño del hardware, se reducen los costos de producción y se facilita la optimización de los sistemas digitales. Además, el uso de compuertas universales contribuye a mejorar la confiabilidad y escalabilidad de los circuitos electrónicos empleados en computadoras, dispositivos móviles y sistemas embebidos.

7. Conclusiones

- Comprendimos la relación directa entre la lógica proposicional y el álgebra de Boole, logrando comprobar de forma grupal cómo los enunciados lógicos se traducen perfectamente a variables binarias (0 y 1) mediante tablas de verdad.
- Aplicamos con éxito las operaciones booleanas fundamentales (AND, OR, NOT, XOR) en nuestro ejercicio del sumador, demostrando cómo estas compuertas básicas se combinan para dar solución a un problema aritmético.

- Dominamos el uso de las técnicas de simplificación algebraica al reducir las expresiones originales del sumador a sus formas más compactas, confirmando que optimizar las ecuaciones disminuye el uso de recursos y compuertas.
- Diseñamos de forma sistemática las funciones lógicas de la Suma y el Acarreo partiendo desde el método SOP (Suma de Productos), lo que nos permitió estructurar el circuito de manera ordenada asegurando que cumpla con los requerimientos.
- Relacionamos el trabajo práctico con la computación real, concluyendo en equipo que el bloque sumador desarrollado es la base física de cómo los procesadores de las computadoras realizan operaciones matemáticas a bajo nivel.

8. Reflexión crítica

a) Edgar Fabricio Rios Cuenca

En esta práctica comprendí la relación del álgebra de Boole con el diseño digital al simplificar funciones mediante tablas de verdad y leyes lógicas. Apliqué el teorema de De Morgan para implementar circuitos optimizados usando solo compuertas universales (NAND/NOR) y diseñé con éxito un sumador binario de 1 bit, logrando traducir la lógica abstracta en hardware real.

b) Jose Camilo Merino Morocho

Esta práctica me demostró que el diseño digital va más allá de aplicar matemáticas; es un reto de optimización real. Al simplificar las ecuaciones algebraicas del sumador, reduje drásticamente el número de compuertas necesarias. Esto me enseñó que una buena simplificación lógica es clave en la ingeniería de hardware, ya que un circuito con menos componentes es más rápido, genera menos calor y reduce costos de fabricación.

c) Ashley Patricia Mocha Valarezo

Las compuertas NAND y NOR son muy importantes en el diseño digital. Con ellas, podemos crear cualquier circuito lógico, lo que las hace muy útiles y baratas. Los teoremas de De Morgan nos ayudan a cambiar expresiones para trabajar solo con estas compuertas. De esta manera, los circuitos se vuelven más simples.

d) Abraham Daniel Maza Lapo

En esta práctica logré aterrizar la relación entre el álgebra de Boole y el diseño digital, viendo cómo la teoría se convierte en hardware real. Al aplicar el teorema de De Morgan para optimizar circuitos con compuertas universales, entendí que la eficiencia matemática es clave para reducir componentes físicos. Finalmente, diseñar el sumador binario de 1 bit me permitió traducir lógica abstracta en un sistema funcional, dándome las bases para entender arquitecturas computacionales más complejas.

8. ANEXOS

a) Edgar Fabricio Rios Cuenca

Paso 2:

Ejercicio 1:

Paso 2:

Los estudiantes cuestionan y resuelven de forma Participativa las leyes y propiedades adicionales mediante ejemplos prácticos conectando cada propiedad con aplicaciones reales en diseño digital.

1) Consenso Dual.

$$F = (A+B) (\bar{A}+C) (B+C) (D+C)$$
$$\text{Consenso Dual} = (B+C)$$
$$\text{Eliminar} = (B+C)$$
$$F = (A+B) (\bar{A}+B) (D+C) //$$

Ejercicio 2:

2.- Expansión de Shannon.

Función $\rightarrow F(A, B, C) = AB + A\bar{C} + \bar{B}C$

Evaluar $A = 1 \rightarrow F(1, B, C) = (1)B + (1)\bar{C} + \bar{B}C$

$$F(1, B, C) = B + \bar{C} + \bar{B}C$$
$$F(1, B, C) = B + C + \bar{C}$$
$$F(1, B, C) = 1$$

Evaluar $A = 0 \rightarrow F(0, B, C) = (0)B + (0)\bar{C} + \bar{B}C$

$$F(0, B, C) = \bar{B}C$$

Resultado Final $\rightarrow F = A(1) + \bar{A}(\bar{B}C)$

$$F = A + \bar{A}\bar{B}C$$
$$F = A + \bar{B}C //$$

Ejercicio 3:

3. Resolución Booleana

- ▶ Clausula 1: $x + y$
(o es un problema de software, o es de hardware).
- ▶ Clausula 2: $\bar{x} + y$
(Si no es un problema de Software, entonces se reinicia).
- ▶ Clausula 3: $y + w$
(si no es un problema de hardware, entonces se activa la alerta).
- ▶ Clausula 4: $\bar{z}$
(El sistema no se reinicia).
- ▶ Resolvente 1: Entre 2 y 4.
$$\begin{array}{r} (\bar{x} + \bar{z}) \quad (\bar{z}) \\ \hline \bar{x} \end{array}$$
- ▶ Resolvente 2: Entre Resolvente 1 y 1.
$$\begin{array}{r} (x + y) \quad (\bar{x}) \\ \hline y \end{array}$$
- ▶ Resolvente 3: Entre Resolvente 2 y 3.
$$\begin{array}{r} (y + w) \quad (y) \\ \hline w \end{array}$$

$w \rightarrow$ Se activa la alerta

Paso 3:

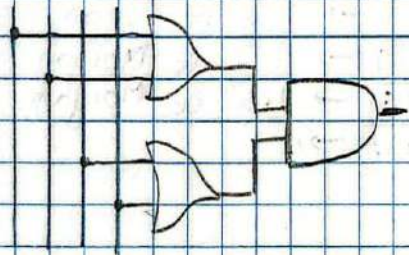
Ejercicio 1:

Ejercicio 1:

Función Original:

$$F = (A+B)(C+D)$$

A B C D



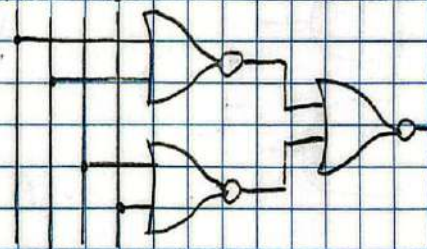
Función final NOR

$$F = \overline{(A+B)(C+D)} \rightarrow \text{Ley de doble Negación.}$$

$$\overline{(A+B)(C+D)} \rightarrow \text{Ley de Morgan,}$$

$$\overline{(A+B)} + \overline{(C+D)}$$

A B C D



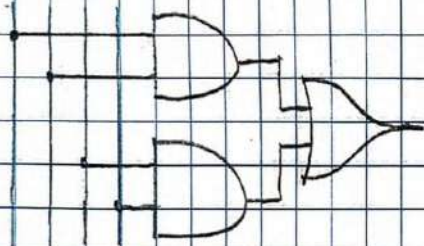
Ejercicio 2:

Ejercicio 2:

Función Original.

$$F = AB + CD$$

A B C D



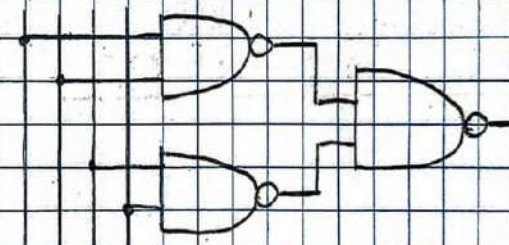
Función final NAND

$$F = \overline{\overline{AB + CD}} \rightarrow \text{Ley de doble Negación}$$

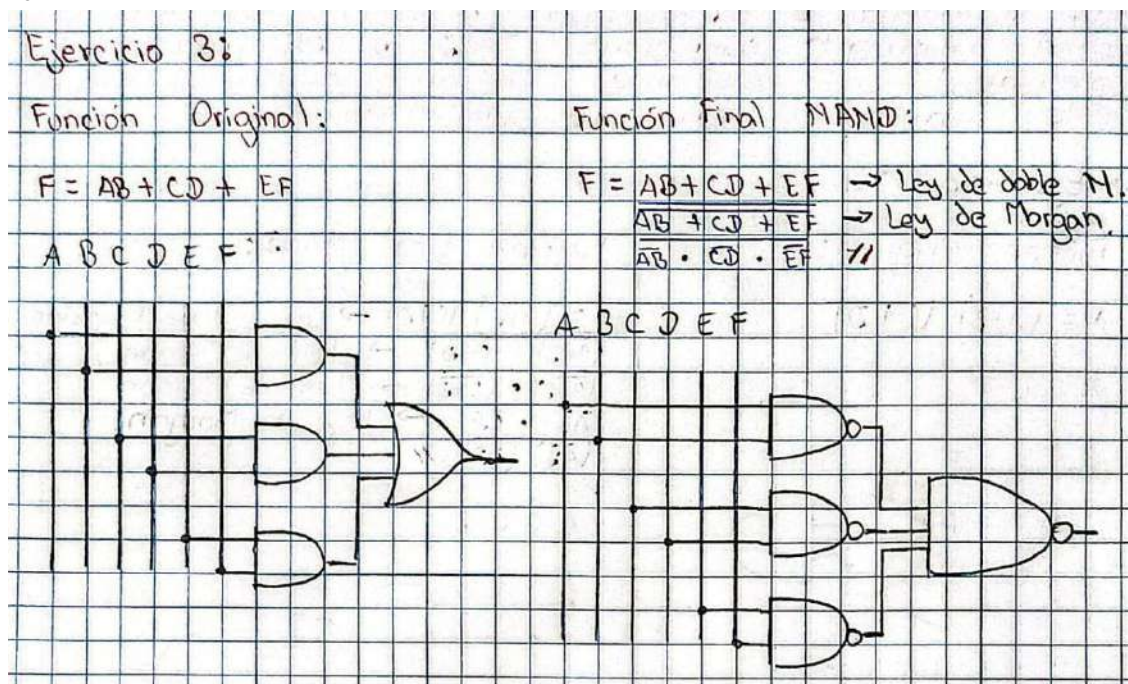
$$\overline{\overline{AB + CD}} \rightarrow \text{Ley de Morgan,}$$

$$\overline{AB} \cdot \overline{CD}$$

A B C D



Ejercicio 3:



Paso 4:

Paso 4:

Los equipos elaboran un informe reflexivo sobre la optimización lógica en sistemas computacionales, analizando como la simplificación booleana reduce costos, consumo energético y huella ambiental en la fabricación de circuitos.

Informe reflexivo sobre la optimización lógica en sistemas computacionales.

En el diseño de sistemas computacionales modernos, la eficiencia ya no se mide únicamente en términos de velocidad de procesamiento, sino también en función de la optimización de los recursos materiales y energéticos. El presente informe expone nuestra reflexión colectiva sobre el impacto de la optimización lógica y la simplificación booleana en la ingeniería de circuitos. A lo largo de este análisis, demostramos cómo las decisiones de diseño matemático en el software abstracto se traducen directamente en beneficios económicos y ecológicos tangibles en el mundo físico.

Análisis de la simplificación booleana.

La simplificación booleana (mediante métodos como los Mapas de Karnaugh) permite reducir al mínimo el número de compuertas lógicas e interconexiones necesarias para ejecutar la función. Este impacta positivamente en tres ejes críticos:

1. Reducción de Costos de fabricación:

- Al reducir las compuertas necesarias, disminuye el área de silicio requerida para cada circuito integrado. Esto permite crear más microchips por cada oblea de producción.
- Los circuitos más pequeños y menos complejos tienen una menor tasa de defectos físicos durante el proceso de fotolitografía, lo que reduce el desperdicio de materiales costosos y maximizar el rendimiento (yield) de la fábrica.

2. Consumo Energético:

- Cada compuerta lógica consume energía eléctrica y disipa calor cada vez que cambia de estado. Eliminar componentes redundantes reduce la capacitación global y las corrientes de fuga del circuito.
- Un menor consumo de energía disminuye la temperatura operativa del dispositivo, reduciendo o eliminando la necesidad de sistemas de enfriamiento activos (como ventiladores), lo que genera un ahorro energético secundario.

3. Huella Ambiental:

- Circuitos más pequeños significan una menor demanda de materias primas críticas, como silicio de alta pureza, cobre y tierras raras, mitigando el impacto de la minería intensiva.
- La reducción del tamaño de los componentes disminuye el uso de químicos peligrosos en las etapas de grabado (etching) y lavado en las plantas de semiconductores.
- A largo plazo, millones de dispositivos optimizados operando globalmente reducen de manera significativa la demanda eléctrica y las emisiones indirectas de CO_2 .

Conclusión Reflexiva.

Como equipo, concluimos que la optimización de sistemas lógicos no es solo un ejercicio de eficiencia técnica, sino una práctica de ingeniería responsable. La simplificación de una ecuación booleana en la fase de diseño de software tiene un impacto en el ecosistema, demostrando que la optimización matemática es una de las herramientas más limpias para reducir costos industriales y mitigar la huella ecológica de la era digital.

Paso 5:

5.- Caso aplicado a la computación: Sumador binario de 4 bit

1: Tabla de verdad.

Entrada: A, B y Acarreo de entrada (C_{in})

Salida: Suma (S) y Acarreo de Salida (C_{out}):

A	B	C_{in}	S	C_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

2.- Simplificación de la Suma.

SOP: $S = \bar{A}\bar{B}C_{in} + \bar{A}B\bar{C}_{in} + A\bar{B}\bar{C}_{in} + ABC$

$\bar{A}\bar{B}C_{in} + \bar{A}B\bar{C}_{in} + A\bar{B}\bar{C}_{in} + ABC \rightarrow$ Ley Distributiva

$\bar{A}(\bar{B}C_{in} + B\bar{C}_{in}) + A(\bar{B}\bar{C}_{in} + BC_{in}) \rightarrow$ Sustitución

$\bar{A}(B \oplus C_{in}) + A(B \oplus C_{in}) \rightarrow$ Definimos variable auxiliar. $B \oplus C_{in} = X$

$\bar{A}X + AX \rightarrow$ Sustitución

$A \oplus X \rightarrow$ Reemplazo del auxiliar.

$A \oplus B \oplus C_{in}$

(Para sustituciones definimos).

Definición de XOR:

$X \oplus Y = \bar{X}Y + X\bar{Y}$

Definición de XNOR

$X \odot Y = \overline{X \oplus Y} = \bar{X}\bar{Y} + XY$

3. Simplificación de salida de acarreo (Cout)

$$SOP = Cout = \bar{A}BC_{in} + A\bar{B}C_{in} + AB\bar{C}_{in} + ABC_{in}$$

$$\bar{A}BC_{in} + A\bar{B}C_{in} + AB\bar{C}_{in} + ABC_{in} \rightarrow \text{Ley distributiva.}$$

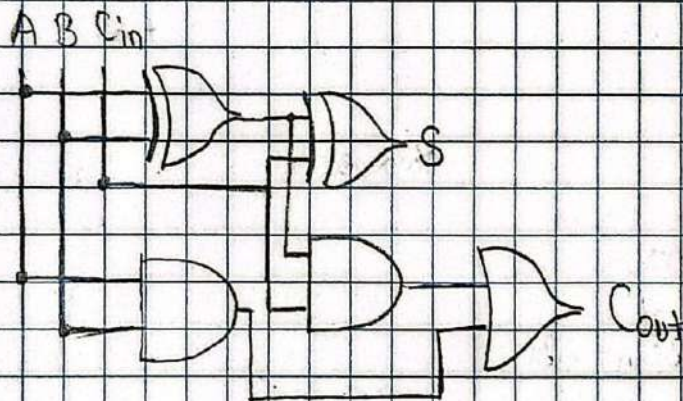
$$C_{in} (\bar{A}B + A\bar{B}) + AB (\bar{C}_{in} + C_{in}) \rightarrow \text{Ley de Complemento}$$

$$C_{in} (\bar{A}B + A\bar{B}) + AB (1) \rightarrow \text{Ley de identidad.}$$

$$C_{in} (\bar{A}B + A\bar{B}) + AB \rightarrow \text{Sustitución.}$$

$$C_{in} (A \oplus B) + AB$$

4. Diseño de las compuertas lógicas.



b) Jose Camilo Merino Morocho

Paso 2:

Ejercicio 1:

Paso N°2
Los estudiantes investigan y presentan de Forma Participativa las leyes y propiedades adicionales mediante ejemplos prácticos, conectando cada propiedad con aplicaciones reales en diseño digital

1) Consenso Dual

$$F = (A+B)(\bar{A}+C)(B+C)(D+C)$$
$$\text{Consenso} = (B+C)$$
$$\text{Eliminar} = (B+C)$$
$$F = (A+B)(\bar{A}+C)(D+C) //$$

Ejercicio 2:

2) Expansion De Shanon

Funcion $\rightarrow F(A,B,C) = AB + A\bar{C} + \bar{B}C$

Evaluar $A=1 \rightarrow F(1,B,C) = (1)B + (1)\bar{C} + \bar{B}C$
 $F(1,B,C) = B + \bar{C} + \bar{B}C$
 $F(1,B,C) = 1$

Evaluar $A=0 \rightarrow F(0,B,C) = (0)B + (0)\bar{C} + \bar{B}C$
 $= \bar{B}C$

Resultado final $\rightarrow F = A(1) + \bar{A}(\bar{B}C)$
 $F = A + \bar{A}\bar{B}C$
 $F = A + \bar{B}C$

Ejercicio 3:

3) Resolución Booleana

- ▶ Clausula 1: $X + Y$
(O es un problema de Software, o es de hardware)
- ▶ Clausula 2: $\bar{X} + Y$
(Si no es un problema de Software, entonces se reinicia)
- ▶ Clausula 3: $\bar{Y} + W$
(Si es un problema de hardware, entonces se activa la alerta)
- ▶ Clausula 4: $\bar{Z}$
(El Sistema no se reinicia)

- ▶ Resolvente 1: Entre 2 y 4

$$(\bar{X} + Z)(\bar{Z}) = \bar{X}$$

- ▶ Resolvente 2: Entre Resolvente 1 y 1

$$(X + Y)(\bar{X}) = Y$$

- ▶ Resolvente 3: Entre Resolvente 2 y 3

$$(\bar{Y} + W)(Y) = W$$

$W \rightarrow$ Se activa la alerta

Paso 3:

Ejercicio 1:

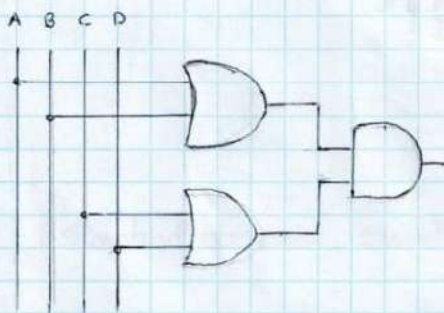
Paso N° 3:

Practica de implementación de funciones booleanas usando únicamente compuertas NAND o NOR (puertas universales): los estudiantes diseñan circuitos equivalentes usando solo un tipo de compuerta

• Ejercicio 1

- Funcion Original

$$F = (A+B)(C+D)$$

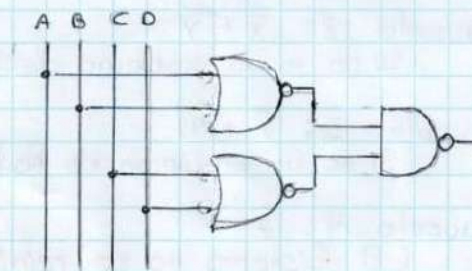


Funcion Final NAND

$$F = (A+B)(C+D) \rightarrow \text{L. Doble Negacion}$$

$$\overline{F} = \overline{(A+B)(C+D)} \rightarrow \text{L. De Morgan}$$

$$\overline{F} = \overline{(A+B)} + \overline{(C+D)}$$



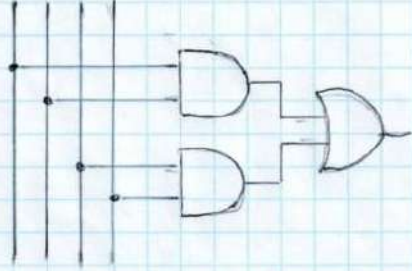
Ejercicio 2:

• Ejercicio 2

Funcion Original

$$F = AB + CD$$

A B C D



Funcion Final NOR

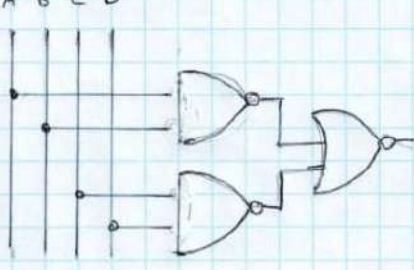
$$F = AB + CD \rightarrow \text{L. de Doble Negacion}$$

$$\overline{F} = \overline{AB + CD} \rightarrow \text{L. De Morgan}$$

$$F = \overline{\overline{AB + CD}} \rightarrow \text{L. De Morgan}$$

$$F = \overline{\overline{AB} \cdot \overline{CD}} \rightarrow \text{L. De Morgan}$$

A B C D



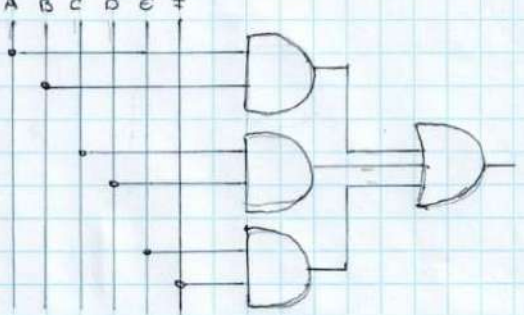
Ejercicio 3:

• Ejercicio 3

Funcion Original

$$F = AB + CD + EF$$

A B C D E F



Funcion Final

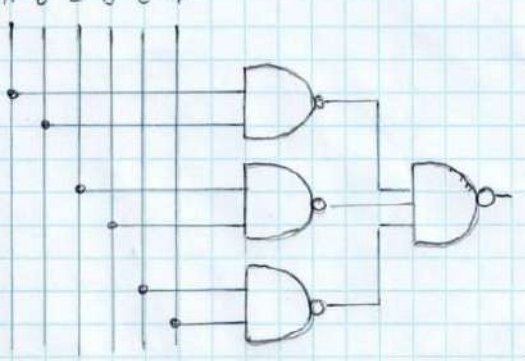
$$F = AB + CD + EF$$

$$\overline{F} = \overline{AB + CD + EF} \rightarrow \text{L. Doble Negacion}$$

$$F = \overline{\overline{AB + CD + EF}} \rightarrow \text{L. De Morgan}$$

$$F = \overline{\overline{AB} \cdot \overline{CD} \cdot \overline{EF}}$$

A B C D E F



Paso 4:

Paso 4: Los equipos elaboran un informe reflexivo sobre la optimización lógica en sistemas computacionales, analizando cómo la simplificación booleana reduce costos, consumo energético y huella ambiental en la fabricación de circuitos.

Informe reflexivo sobre la Optimización Lógica en Sistemas Computacional

En el diseño de Sistemas Computacionales modernos la eficiencia ya no se mide únicamente en términos de velocidad de procesamiento sino también en función de la optimización de los recursos materiales y energéticos. El presente informe expone nuestra reflexión colectiva sobre el impacto de la optimización lógica y la simplificación booleana en la Ingeniería de circuitos. A lo largo de este análisis, demostramos cómo las decisiones de diseño matemático en el software abstracto se traducen directamente en beneficios económicos y ecológicos tangibles en el mundo físico.

Análisis de la Simplificación Booleana

La Simplificación booleana (mediante métodos como los mapas de Karnaugh) permite reducir al mínimo el número de compuertas lógicas e interconexiones necesarias para ejecutar la función. Este impacto positivamente en tres críticos:

1. Reducción de Costos de Fabricación

- ▶ Al reducir las compuertas necesarias disminuye el área de silicio requerido para cada circuito integrado. Esto permite crear más microchips por cada oblea de producción.
- ▶ Los circuitos más pequeños y menos complejos tienen una menor tasa de defectos físicos durante el proceso de fotolitografía, lo cual reduce el desperdicio de materiales costosos y maximizar el rendimiento (yield) de la fábrica.

2. Consumo Energético

- ▶ Cada compuerta lógica consume energía eléctrica y disipa calor cada vez que cambia de estado. Eliminar componentes redundantes reduce la capacitación global y las corrientes de fuga de circuito.

- ▶ Un menor consumo de energía disminuye la temperatura operativa del dispositivo, reduciendo o eliminando la necesidad de sistemas de enfriamiento activos (como ventiladores) lo que genera un ahorro energético secundario

3. Huella Ambiental

- ▶ Circuitos más pequeños significan una menor demanda de materiales primos críticos como silicio de alta pureza, cobre y tierras raras mitigando el impacto de la minería intensiva
- ▶ La reducción del tamaño de los componentes disminuye el uso de químicos peligrosos en las etapas de grabado (etching) y lavado en las plantas de semiconductores
- ▶ A largo plazo, millones de dispositivos optimizados operando globalmente reducen de manera significativamente la demanda eléctrica y las emisiones indirectas de CO_2

Conclusion Reflexiva

Como equipo, concluimos que la optimización de sistemas lógicos no es solo ejercicio de eficiencia técnica sino una práctica de ingeniería responsable. La simplificación de una ecuación booleana en la fase de diseño de software tiene un impacto en el ecosistema demostrando que la optimización matemática es una de las herramientas más limpias para reducir costos industriales y mitigar la huella ecológica de la era digital.

Paso 5:

Paso N° 5
Caso Aplicado a la Computación: Suma binaria 1 bit

1) Tabla De Verdad

Entradas: A, B, Acorreo De Entrada (Cin)

Salidas: Suma (S), Acorreo De Salida (Cout)

A	B	Cin	S	Cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Para las Sustituciones

Definición de XOR:

$$X \oplus Y = \bar{X}Y + X\bar{Y}$$

Definición de XNOR:

$$X \odot Y = \overline{X \oplus Y} = \bar{X}\bar{Y} + XY$$

2) Simplificación De la Suma

2.1 - SOP

$$S = \bar{A}\bar{B}C_{in} + \bar{A}B\bar{C}_{in} + A\bar{B}\bar{C}_{in} + ABC$$

$$\bar{A}\bar{B}C_{in} + \bar{A}B\bar{C}_{in} + A\bar{B}\bar{C}_{in} + ABC \rightarrow \text{Ley Distributiva}$$

$$A(\bar{B}C_{in} + \bar{C}_{in}) + A(B\bar{C}_{in} + C_{in}) \rightarrow \text{Sustitucion}$$

$$\bar{A}(\bar{B} \oplus C_{in}) + A(B \oplus C_{in}) \rightarrow \text{Definimos Variable } B \oplus C_{in} = X$$

$$\bar{A}X + AX \rightarrow \text{Sustitucion}$$

$$\bar{A} \oplus X \rightarrow \text{Reemplazo Auxiliar}$$

$$A \oplus B \oplus C_{in} //$$

3) Simplificación De la Salida de Acorreo (Cout)

3.1 - SOP

$$C_{out} = \bar{A}BC_{in} + \bar{A}B\bar{C}_{in} + A\bar{B}C_{in} + ABC_{in}$$

$$\bar{A}BC_{in} + \bar{A}B\bar{C}_{in} + A\bar{B}C_{in} + ABC_{in} \rightarrow \text{Ley Distributiva}$$

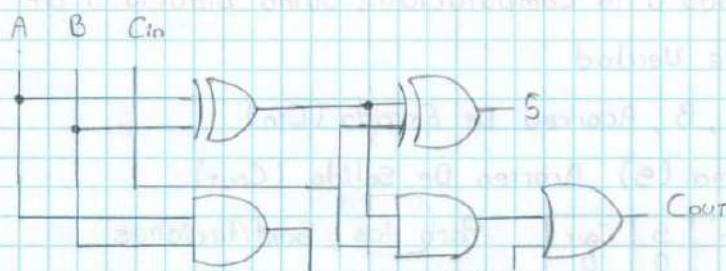
$$C_{in}(\bar{A}B + A\bar{B}) + AB(\bar{C}_{in} + C_{in}) \rightarrow \text{Ley de Complemento}$$

$$C_{in}(\bar{A}B + A\bar{B}) + AB(1) \rightarrow \text{Ley de Identidad}$$

$$C_{in}(\bar{A}B + A\bar{B}) + AB \rightarrow \text{Sustitucion}$$

$$C_{in}(A \oplus B) + AB$$

4) Diseño De Componentes Logicos



c). Ashley Patricia Mocha Valarezo

Ejercicio 1:

Universidad Nacional de Loja

Nombre: Ashley Patricia Mocha Valarezo

Unidad: 8

Título: APE 3

Paso 2:

Los estudiantes investigan y presentan de forma participativa las leyes y propiedades adicionales mediante ejemplos prácticos, conectando cada propiedad con aplicaciones reales en diseño digital.

① Consenso Dual

$$F = (A+B)(\bar{A}+C)(B+C)(D+C)$$

Consenso = $(B+C)$

El minor = $(B+C)$

$$F = (A+B)(\bar{A}+C)(D+C)$$

Ejercicio 2:

② Expansión de Shannon

$$F(A, B, C) = AB + \bar{A}\bar{C} + \bar{B}C$$

Evaluar $A=1 \rightarrow F(1, B, C) = (1)B + (1)\bar{C} + \bar{B}C$

$$F(1, B, C) = B + \bar{C} + \bar{B}C$$

$F(1, B, C) = B + \bar{B}C + \bar{C} \rightarrow$ ley conmutativa

$F(1, B, C) = (B + \bar{B})(B + C) + \bar{C} \rightarrow$ ley distributiva, y ley de complemento

$F(1, B, C) = (1)(B + C) + \bar{C} \rightarrow$ ley de identidad

$F(1, B, C) = B + C + \bar{C} \rightarrow$ ley de complemento

$F(1, B, C) = B + 1 \rightarrow$ ley de identidad

$F(1, B, C) = 1$

Evaluar $A=0 \rightarrow F(0, B, C) = (0)B + (0)\bar{C} + \bar{B}C$

$$F(0, B, C) = \bar{B}C$$

Resultado final $\rightarrow F = A(1) + \bar{A}(\bar{B}C)$

$$F = A + \bar{A}\bar{B}C$$

Resultado final extendido $\rightarrow F = A(B + \bar{C} + \bar{B}C) + \bar{A}\bar{B}C$

Ejercicio 3:

③ Resolución booleana

- Clausula 1: $x + y$
(0 es un problema de software o de hardware)
- Clausula 2: $\bar{x} + y$
(Si no es un problema de software, entonces se reinicia).
- Clausula 3: $\bar{y} + w$
(Si es un problema de hardware, entonces se activa la alerta)
- Clausula 4: $\bar{z}$
(El sistema no se reinicia)
- Resolvente 1: Entre 2 y 4.
 $(\bar{x} + z)(\bar{z})$
 $\bar{x}$
- Resolvente 2: Entre resolvente 1 y 1.
 $(x + y)(\bar{x})$
 y
- Resolvente 3: Entre resolvente 2 y 3.
 $(\bar{y} + w)(y)$
 $w \Rightarrow$ Se activa la alerta.

Paso 3:

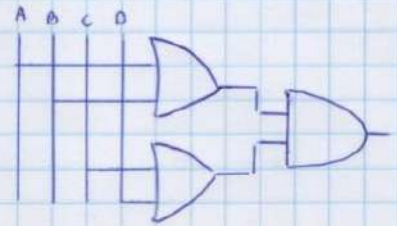
Ejercicio 1:

Paso 3: Práctica de implementación de funciones booleanas usando únicamente compuertas NAND o NOR (puertas universales). Los estudiantes diseñan circuitos equivalentes usando un tipo de compuertas.

- Ejercicio 1: $F = (A+B)(C+D)$

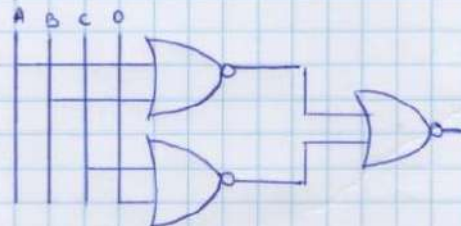
Función original

$F = (A+B)(C+D)$

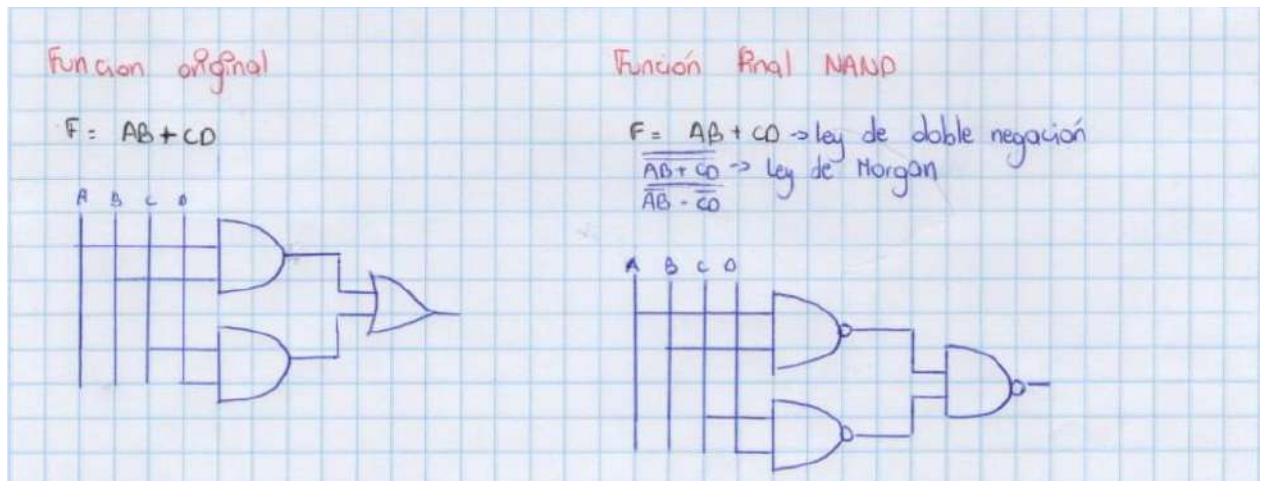


Función final NOR

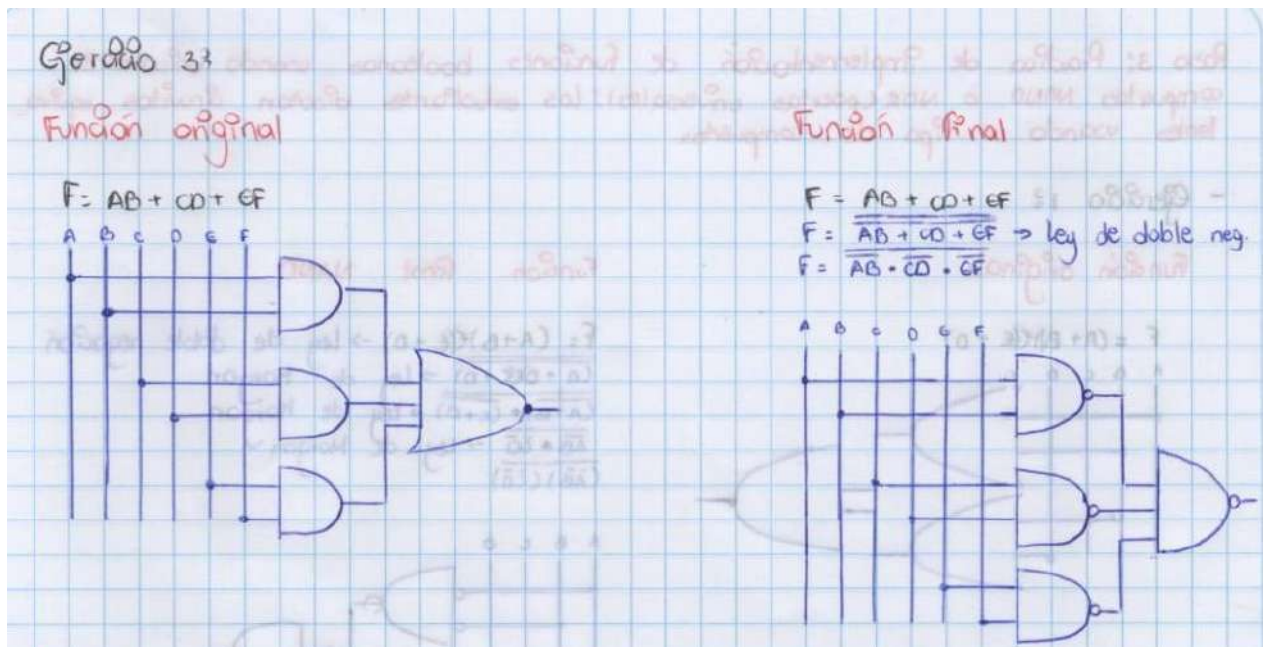
$F = (A+B)(C+D) \rightarrow$ Ley de doble negación
 $\overline{\overline{(A+B)(C+D)}} \rightarrow$ Ley de Morgan
 $\overline{(\overline{A+B}) + (\overline{C+D})}$



Ejercicio 2:



Ejercicio 3:



Paso 4:

Paso 4:

Los equipos elaboran un informe reflexivo sobre la optimización lógica en sistemas computacionales, analizando cómo la simplificación booleana reduce costos, consumo energético y huella ambiental en la fabricación de circuitos.

Informe reflexivo sobre la optimización lógica en sistemas computacionales.

En el diseño de sistemas computacionales modernos, la eficiencia ya no se mide únicamente en términos de velocidad de procesamiento, sino también en función de la optimización de los recursos materiales y energéticos.

El presente informe expone nuestra reflexión colectiva sobre el impacto de la optimización lógica y la simplificación booleana en la ingeniería de circuitos. A lo largo de este análisis, demostramos cómo las decisiones de diseño matemático en el software abstracto se traducen directamente en beneficios económicos y ecológicos tangibles en el mundo físico.

Análisis de simplificación booleana

La simplificación booleana (mediante métodos como los Mapas de Karnaugh) permite reducir al mínimo el número de compuertas lógicas e interconexiones necesarias para ejecutar la función. Este impacto positivamente en tres críticas:

1. Reducción de Costos de Fabricación:

- Al reducir las compuertas necesarias, disminuye el área de silicio requerida para cada circuito integrado. Esto permite crear más microchips por cada oblea de producción.
- Los circuitos más pequeños y menos complejos tienen una menor tasa de defectos físicos durante el proceso de fotografía, lo que reduce el desperdicio de materiales costosos y maximiza el rendimiento (yield) de la fábrica.

2. Consumo Energético:

- Cada compuerta lógica consume energía eléctrica y disipa calor cada vez que cambia de estado. Eliminar componentes redundantes reduce la capacitancia global y las corrientes de fuga del circuito.
- Un menor consumo de energía disminuye la temperatura operativa del dispositivo, reduciendo o eliminando la necesidad de sistemas de enfriamiento activo (como ventiladores), lo que genera un ahorro de energía secundaria.

3. Huella Ambiental:

- Circuitos más pequeños significaban una menor demanda de materias primas críticas, como silicio de alta pureza, cobre y tierras raras, mitigando el impacto de la minería intensiva.
- La reducción del tamaño de los componentes disminuyen el uso de químicos peligrosos en las etapas de grabado (etching) y lavado en las plantas.

de semiconductores

- A largo plazo, millones de dispositivos optimizados operando globalmente reducen de manera significativa la demanda eléctrica y las emisiones indirectas de CO₂.

Conclusion Reflexiva

Como equipo, concluimos que la optimización de sistemas lógicos no es solo un ejercicio de eficiencia técnica, sino una práctica de ingeniería responsable. La simplificación de una educación booleana en la fase de diseño de software tiene un impacto en el ecosistema, demostrando que la optimización matemática es una de las herramientas más limpias para reducir costos industriales y mitigar la huella ecológica de la era digital.

Paso 5:

Paso 5: Caso aplicado a la computación: sumador binario de 1 bit

1. Tabla de verdad

Entradas: A, B y acarreo de la entrada (C_{in})

Salidas: S (suma) y acarreo de la salida (C_{out})

A	B	C _{in}	S	C _{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

2. Simplificación de la Suma

2.1. SOP:

$$S = \bar{A}\bar{B}C_{in} + \bar{A}B\bar{C}_{in} + A\bar{B}\bar{C}_{in} + ABC_{in}$$

$$\bar{A}\bar{B}C_{in} + \bar{A}B\bar{C}_{in} + A\bar{B}\bar{C}_{in} + ABC_{in} \rightarrow \text{Ley Distributiva}$$

$$\bar{A}(\bar{B}C_{in} + B\bar{C}_{in}) + A(\bar{B}\bar{C}_{in} + BC_{in}) \rightarrow \text{XOR definición}$$

$$\bar{A}(B \oplus C_{in}) + A(\bar{B} \oplus C_{in}) \rightarrow \text{XNOR definición}$$

$$\bar{A}(B \oplus C_{in}) + A(\overline{B \oplus C_{in}}) \rightarrow \text{XNOR = complemento de XOR}$$

$$\bar{A}(B \oplus C_{in}) + A(\overline{B \oplus C_{in}}) \rightarrow \text{definición XOR}$$

$$A \oplus (B \oplus C_{in})$$

$$A \oplus B \oplus C_{in}$$

Definiciones:

$$\text{XOR} = X \oplus Y = \bar{X}Y + X\bar{Y}$$

$$\text{XNOR} = X \odot Y = \bar{X}\bar{Y} + XY$$

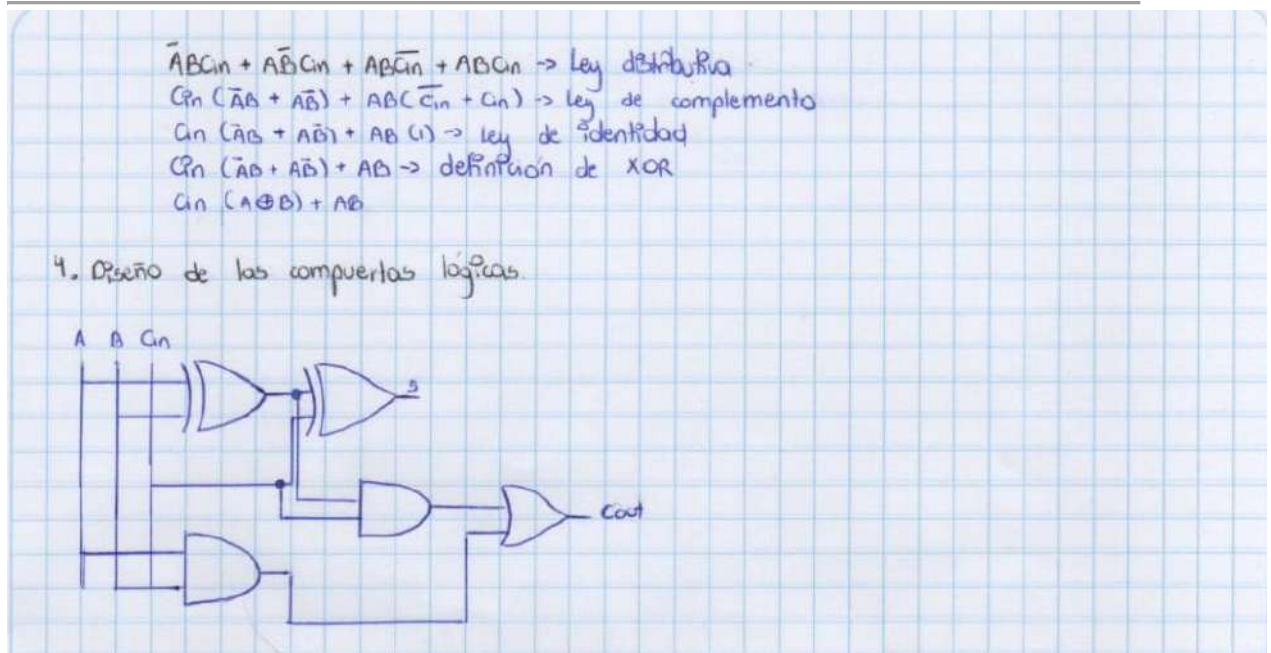
XNOR es complemento de XOR
por lo tanto se puede escribir
la función como:

$$\overline{(X \oplus Y)} = X \odot Y$$

3. Simplificación de la salida de acarreo (C_{out})

3.1. SOP:

$$C_{out} = \bar{A}BC_{in} + \bar{A}B\bar{C}_{in} + A\bar{B}\bar{C}_{in} + ABC_{in}$$



c) Abraham Daniel Maza Lapo

Paso 2:

- Ejercicio 1

1. Consenso dual

$$F = (A+B)(\bar{A}+B)(B+C)(D+C)$$

Consenso = $(B+C)$

Eliminar = $(B+C)$

$$F = (A+B)(\bar{A}+C)(D+C)$$

- Ejercicio 2

2. Expansión de Shannon

$$F(A, B, C) = AB + A\bar{C} + \bar{B}C$$

Evaluar $A=1$ $F(1, B, C) = (1)B + (1)\bar{C} + \bar{B}C$
 $F(1, B, C) = B + \bar{C} + \bar{B}C$
 $F(1, B, C) = 1$

Evaluar $A=0$ $F(0, B, C) = (0)B + (0)\bar{C} + \bar{B}C$
 $= \bar{B}C$

Resultado final $F = A(1) + \bar{A}(\bar{B}C)$
 $F = A + \bar{A}\bar{B}C$
 $F = A + \bar{B}C$

- Ejercicio 3

3. Resolución booleana

Clausula 1: $X + Y$ (o es un problema de software o es un problema de hardware)

Clausula 2: $\bar{X} + Y$ (si no es un problema de software, entonces se reinicia)

Clausula 3: $\bar{Y} + W$ (si es un problema de hardware, entonces se activa la alerta)

Clausula 4: $\bar{Z}$ (el sistema no se reinicia)

Resolvente 1: Entre 2 y 4 $\rightarrow (\bar{X} + Z)(\bar{Z})$
 $\bar{X}$

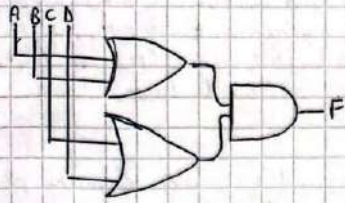
Resolvente 2: Entre resolvente 1 y 1 $(X + Y)(\bar{X})$
 Y

Resolvente 3: Entre resolvente 2 y 3 $(\bar{Y} + W)(Y)$
 W (se activa la alerta)

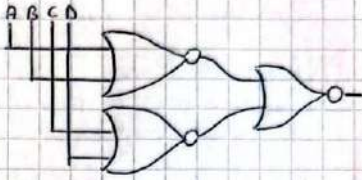
Paso 3:

- Ejercicio 1

Función original
 $F = (A + B)(C + D)$

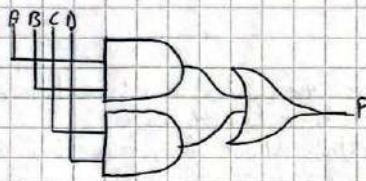


Función Final NOR
 $F = (A + B)(C + D)$ Ley de doble negación
 $\overline{\overline{(A + B)(C + D)}}$ Ley de Morgan
 $\overline{\overline{(A + B)} + \overline{\overline{(C + D)}}}$

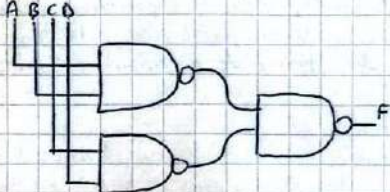


- Ejercicio 2

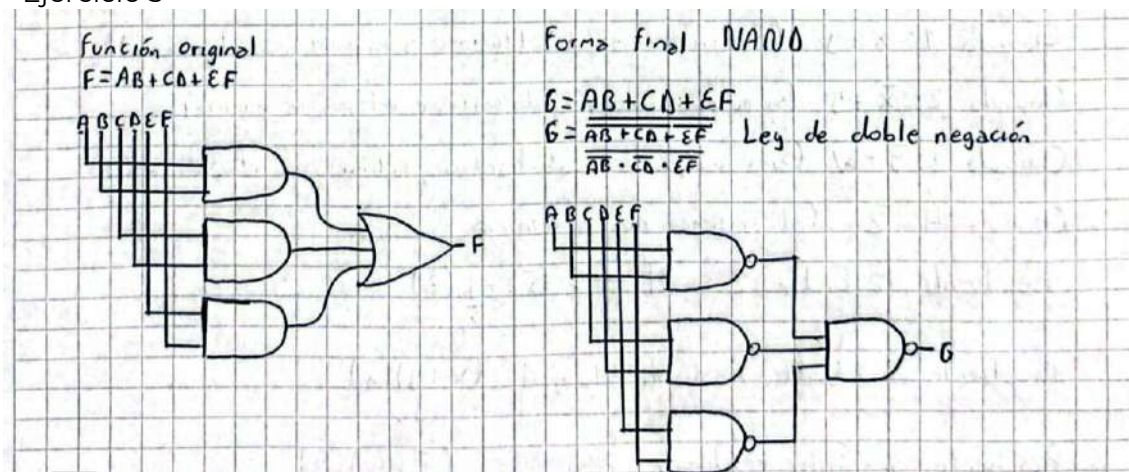
Función original
 $F = AB + CD$



Función Final NAND
 $F = AB + CD$ Ley de doble negación
 $\overline{\overline{AB + CD}}$
 $\overline{\overline{AB} \cdot \overline{\overline{CD}}}$



- Ejercicio 3



Paso 4:

4. Informe sobre la optimización lógica en sistemas computacionales

En el diseño de sistemas computacionales modernos, la eficiencia ya no se mide únicamente en términos de velocidad de procesamiento, sino también en función de la optimización de los recursos materiales y energéticos. El presente informe expone nuestra reflexión colectiva sobre el impacto de la optimización lógica y la simplificación booleana en la ingeniería de circuitos. A lo largo de este análisis, demostramos cómo las decisiones de diseño matemático en el software abstracto se traducen directamente en beneficios económicos y ecológicos tangibles en el mundo físico.

Análisis de la simplificación booleana

Permite reducir al mínimo el número de compuertas lógicas e interconexiones necesarias para ejecutar la función. Este impacto positivamente en tres ejes críticos:

1. Reducción de costos de fabricación

- Al reducir las compuertas necesarias, disminuye el área de silicio requerida para cada circuito integrado. Permitiendo crear más microchips por cada oblea de producción.
- Los circuitos más pequeños y menos complejos tienen una menor tasa de defectos físicos durante el proceso de litografía, lo que reduce el desperdicio de materiales costosos y maximiza el rendimiento de la fábrica.

2. Consumo energético

- Cada compuerta lógica consume energía eléctrica y disipa calor cada vez que cambia de estado. Eliminar componentes redundantes reduce la capacitancia global y las corrientes de fuga del circuito.
- Un menor consumo de energía disminuye la temperatura operativa del dispositivo, reduciendo o eliminando la necesidad de sistemas de enfriamiento activos, lo que genera un ahorro energético secundario.

3. Huella ambiental

- Circuitos más pequeños significan una menor demanda de materias primas críticas, como silicio de alta pureza, cobre y tierras raras, mitigando el impacto de la minería intensiva.
- La reducción del tamaño de los componentes disminuye el uso de químicos peligrosos en las etapas de grabado y lavado en las plantas de semiconductores.
- A la larga, millones de dispositivos optimizados operando globalmente reducen de manera significativa la demanda eléctrica y las emisiones indirectas de CO₂.

Conclusión reflexiva

Como equipo, concluimos que la optimización de sistemas lógicos no es solo un ejercicio de eficiencia técnica, sino una práctica de ingeniería responsable. La simplificación de una ecuación booleana en la fase de diseño de software tiene un impacto en el ecosistema, demostrando que la optimización matemática es una de las herramientas más limpias para reducir costos industriales y mitigar la huella ecológica de la era digital.

Paso 5:

5. Caso aplicado a la computación: sumador binario de 1 bit

1: Tabla de verdad.

Entradas: A, B y Acarreo de entrada (C_{in})

Salidas: Suma (S) y Acarreo de salida (C_{out})

A	B	C_{in}	S	C_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

2: Simplificación de la Suma

2.1. SOP: $S = \bar{A}\bar{B}C_{in} + \bar{A}B\bar{C}_{in} + A\bar{B}\bar{C}_{in} + ABC$

(Para las sustituciones de finidos)

Definición de XOR:
 $X \oplus Y = \bar{X}Y + X\bar{Y}$

$\bar{A}\bar{B}C_{in} + \bar{A}B\bar{C}_{in} + A\bar{B}\bar{C}_{in} + ABC$ Ley distributiva

Definición de XNOR:
 $X \odot Y = \bar{X}\bar{Y} + XY$

$\bar{A}(\bar{B}C_{in} + B\bar{C}_{in}) + A(\bar{B}\bar{C}_{in} + BC_{in})$ Sustitución

$\bar{A}(B \oplus C_{in}) + A(\bar{B} \odot C_{in})$ Definimos variable auxiliar $B \oplus C_{in} = X$

$\bar{A}X + A\bar{X}$ Sustitución

$A \oplus X$ Reemplazo del auxiliar

$A \oplus B \oplus C_{in}$

3. Simplificación de la salida de acarreo (C_{out})

3.1. SOP: $C_{out} = \bar{A}BC_{in} + A\bar{B}C_{in} + AB\bar{C}_{in} + ABC_{in}$

$\bar{A}BC_{in} + A\bar{B}C_{in} + AB\bar{C}_{in} + ABC_{in}$ Ley distributiva

$C_{in}(\bar{A}B + A\bar{B}) + AB(\bar{C}_{in} + C_{in})$ Ley de complemento

$C_{in}(\bar{A}B + A\bar{B}) + AB(1)$ Ley de identidad

$C_{in}(\bar{A}B + A\bar{B}) + AB$ Sustitución

$C_{in}(A \oplus B) + AB$

4. Diseño de las compuertas lógicas

